

Lab 4

Part 1

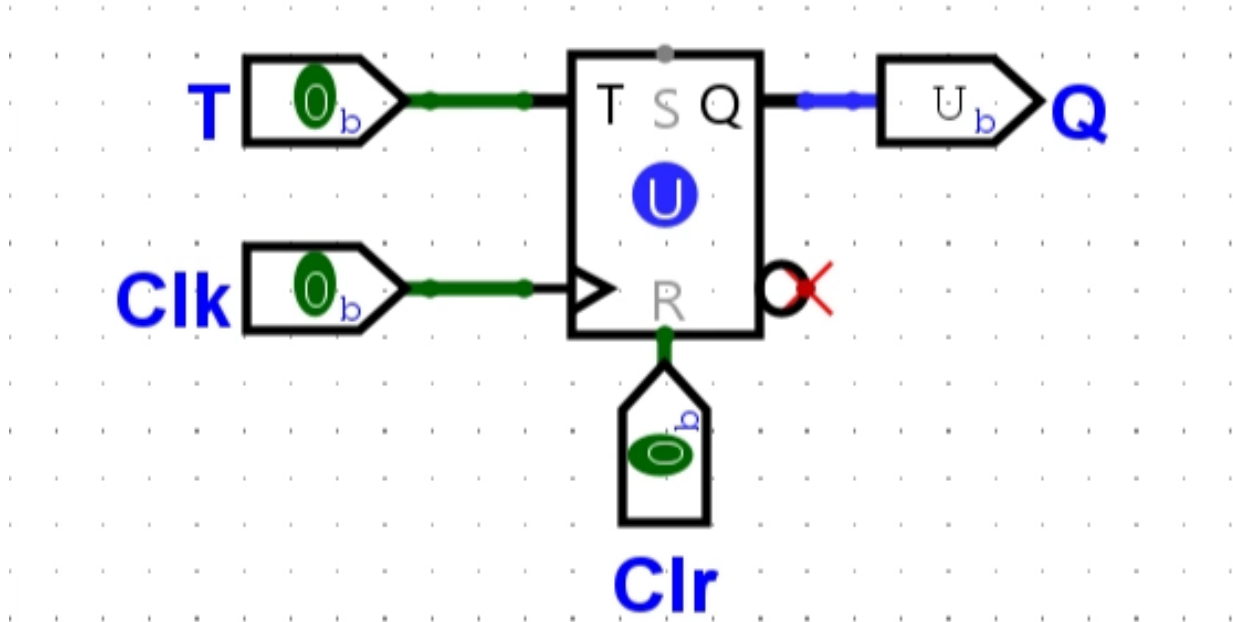


Figure 1 - Logisim T FlipFlop

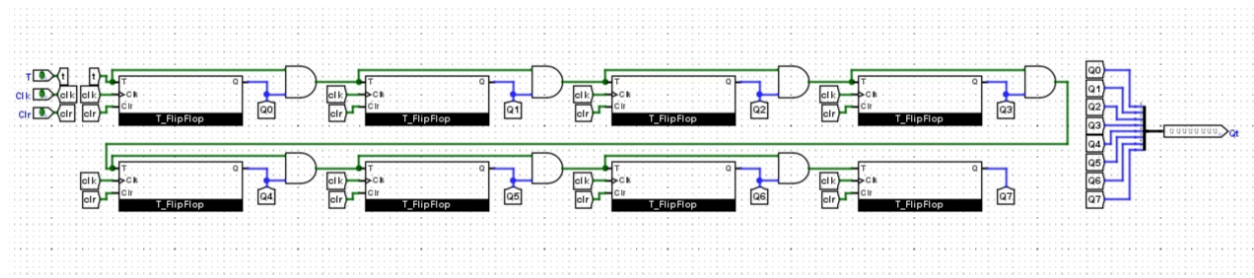


Figure 2 - Logisim 8Bit T FlipFlop

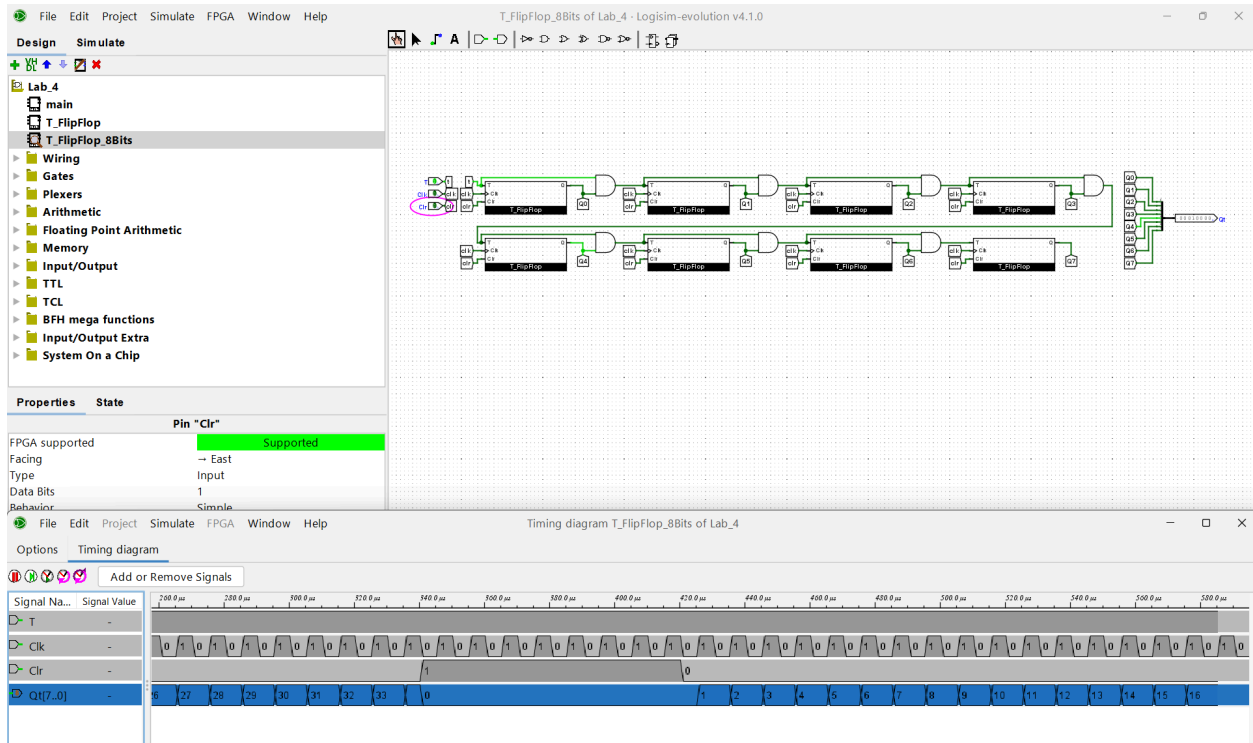


Figure 3 - Timing Diagram of Logisim 8Bit T FlipFlop

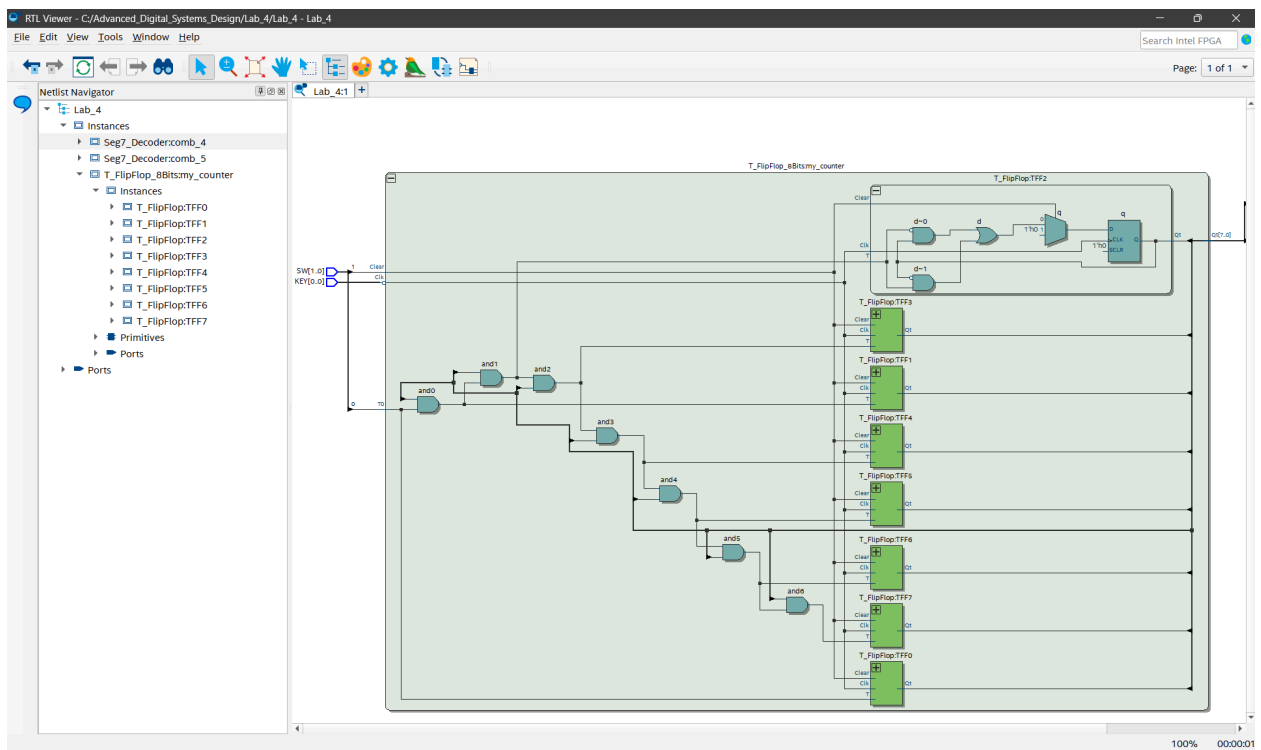


Figure 4 - RTL View of 8Bit T FlipFlop

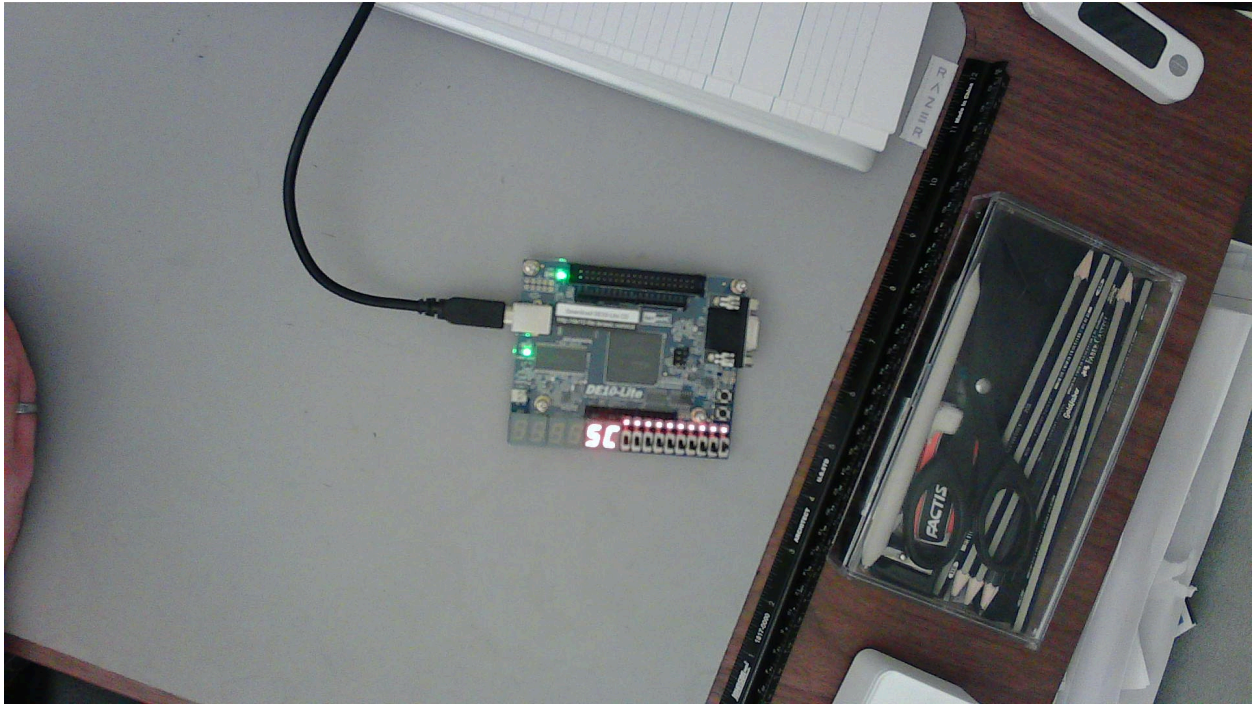


Figure 5 - 7 Segment Counter using 8Bit T FlipFlop instead of 4bits suggested by lab.

```
module T_FlipFlop(  
    input T,  
    input Clk,  
    input Clear,  
    output Qt  
);  
    wire d;  
    reg q;  
    always @ (posedge Clk) begin  
        if(Clear)  
            q <= 0;  
        else  
            q <= d;  
    end  
  
    assign d = (q & ~T) | (T & ~q);  
    assign Qt = q;
```

```
endmodule
```

```
module T_FlipFlop_8Bits(  
    input T0,  
    input Clk,  
    input Clear,  
    output [7:0] Qt  
);  
    wire T1, T2, T3, T4, T5, T6, T7;  
    T_FlipFlop TFF0(  
        .T(T0),  
        .Clk(Clk),  
        .Clear(Clear),  
        .Qt(Qt[0])  
    );  
    and and0 (T1, Qt[0], T0);  
    T_FlipFlop TFF1(  
        .T(T1),  
        .Clk(Clk),  
        .Clear(Clear),  
        .Qt(Qt[1])  
    );  
    and and1 (T2, Qt[1], T1);  
    T_FlipFlop TFF2(  
        .T(T2),  
        .Clk(Clk),  
        .Clear(Clear),  
        .Qt(Qt[2])  
    );  
    and and2 (T3, Qt[2], T2);  
    T_FlipFlop TFF3(  
        .T(T3),  
        .Clk(Clk),  
        .Clear(Clear),
```

```

        .Qt(Qt[3])
    );
    and and3 (T4, Qt[3], T3);
    T_FlipFlop TFF4(
        .T(T4),
        .Clk(Clk),
        .Clear(Clear),
        .Qt(Qt[4])
    );
    and and4 (T5, Qt[4], T4);
    T_FlipFlop TFF5(
        .T(T5),
        .Clk(Clk),
        .Clear(Clear),
        .Qt(Qt[5])
    );
    and and5 (T6, Qt[5], T5);
    T_FlipFlop TFF6(
        .T(T6),
        .Clk(Clk),
        .Clear(Clear),
        .Qt(Qt[6])
    );
    and and6 (T7, Qt[6], T6);
    T_FlipFlop TFF7(
        .T(T7),
        .Clk(Clk),
        .Clear(Clear),
        .Qt(Qt[7])
    );
endmodule

```

Part 2

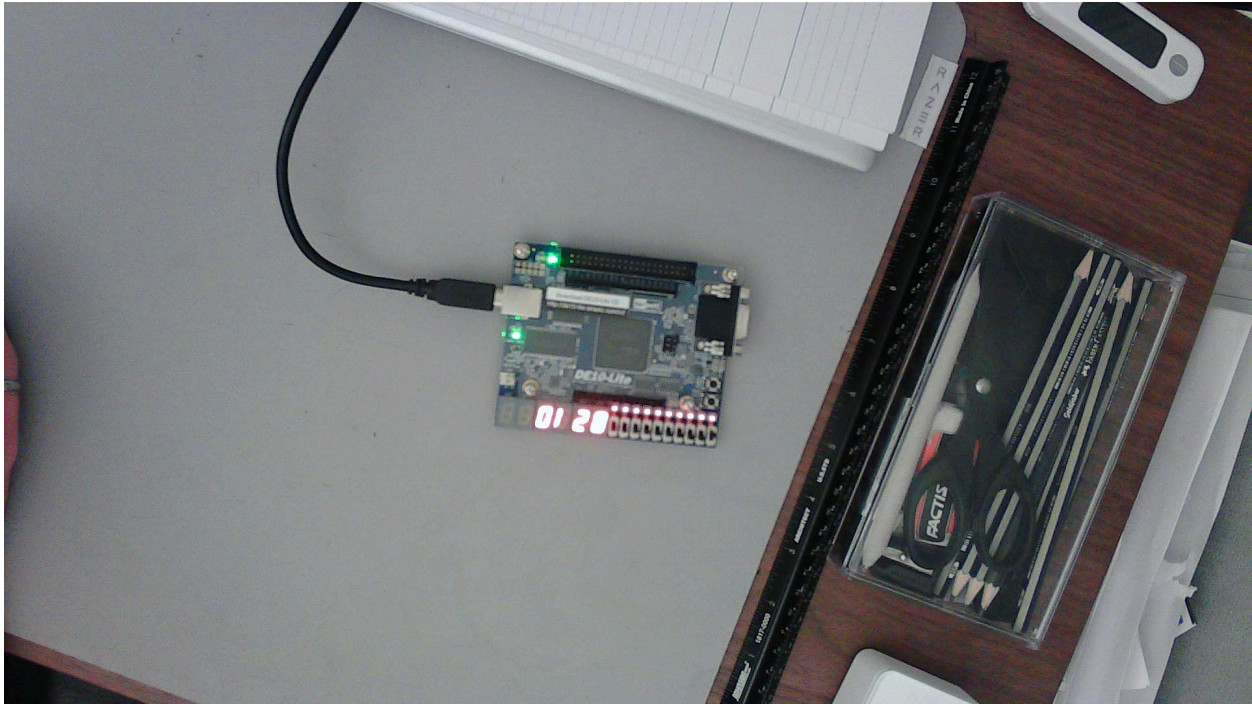


Figure 6 - 16 Bit 7 Segment Counter

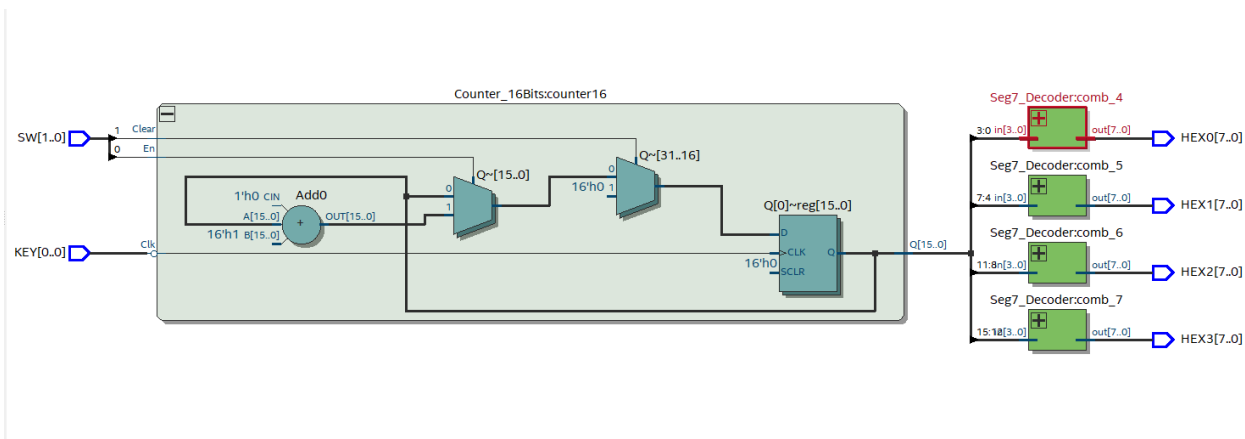


Figure 7 - RTL view of 16 Bit Counter. MUX, FlipFlop and arithmetic units are used.

```
module Counter_16Bits(
    input En,
    input Clk,
    input Clear,
    output reg [15:0] Q
);
    always @ (posedge Clk ) begin
```

```

    if(Clear)
        Q <= 0;
    else
        if(En)
            Q <= Q + 1;
        end
    end
endmodule

```

Part 3

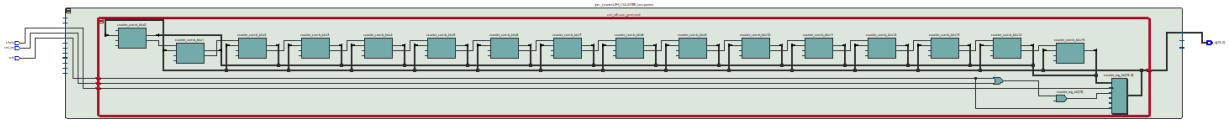


Figure 8 - RTL View of IP Component 16Bit Counter. They use what is called "Combs". Likely because they save you on transistors when the end goal is just counting.

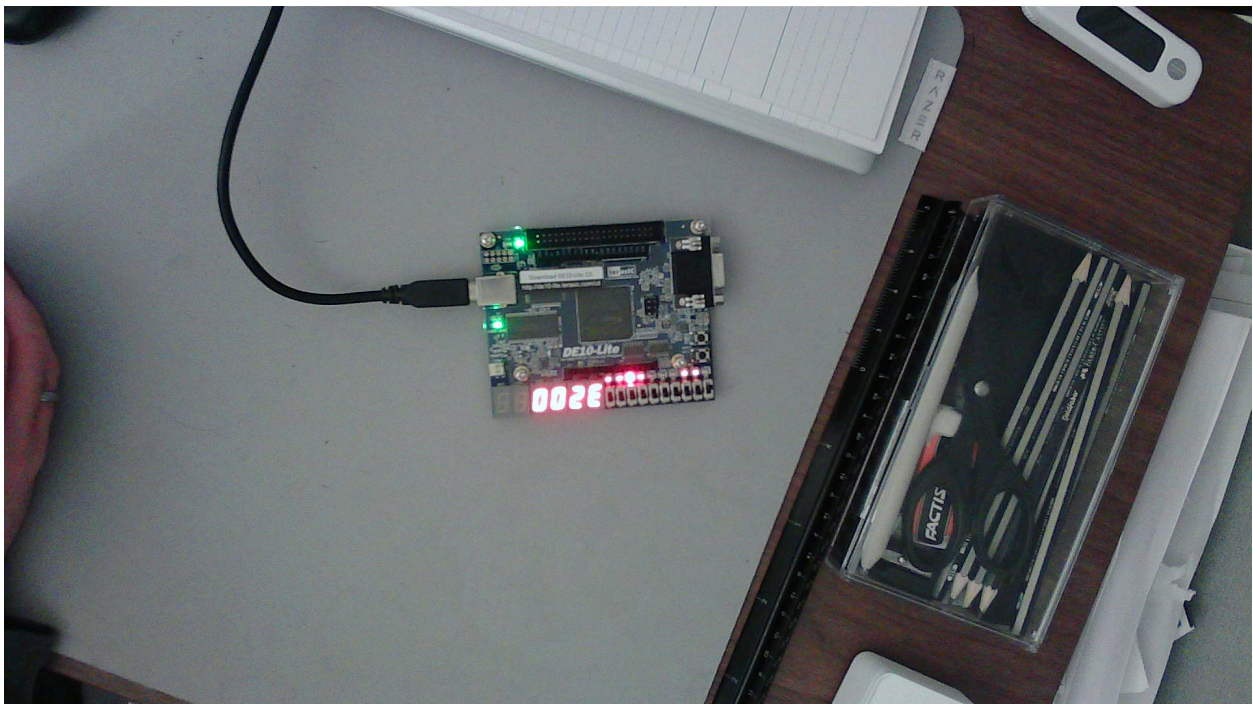


Figure 9 - 16Bit IP Counter. Generated using Quartus IP catalog

```

module LPM_Counter_16Bits (
    clock,
    cnt_en,
    sclr,
    q);

    input    clock;
    input    cnt_en;
    input    sclr;
    output [15:0] q;

    wire [15:0] sub_wire0;
    wire [15:0] q = sub_wire0[15:0];

    lpm_counter LPM_COUNTER_component (
        .clock (clock),
        .cnt_en (cnt_en),
        .sclr (sclr),
        .q (sub_wire0),
        .aclr (1'b0),
        .aload (1'b0),
        .aset (1'b0),
        .cin (1'b1),
        .clk_en (1'b1),
        .cout (),
        .data ({16{1'b0}}),
        .eq (),
        .sload (1'b0),
        .sset (1'b0),
        .updown (1'b1));

    defparam
        LPM_COUNTER_component.lpm_direction = "UP",
        LPM_COUNTER_component.lpm_port_updown = "PORT_UNUSE
D",
        LPM_COUNTER_component.lpm_type = "LPM_COUNTER",

```

```
LPM_COUNTER_component.lpm_width = 16;  
  
endmodule
```

Part 4

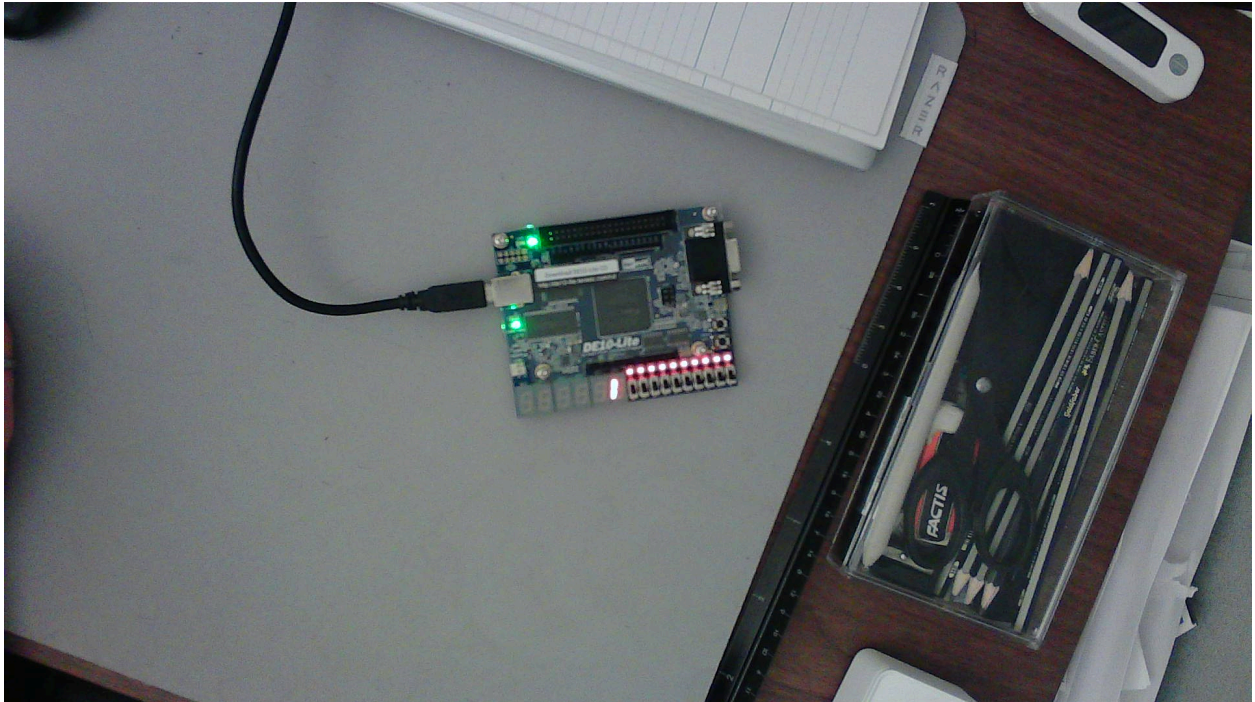


Figure 10 - 50 MHz clock is turned into a 1Hz output by making a modulo 50M counter that outputs once at the end of the counter.

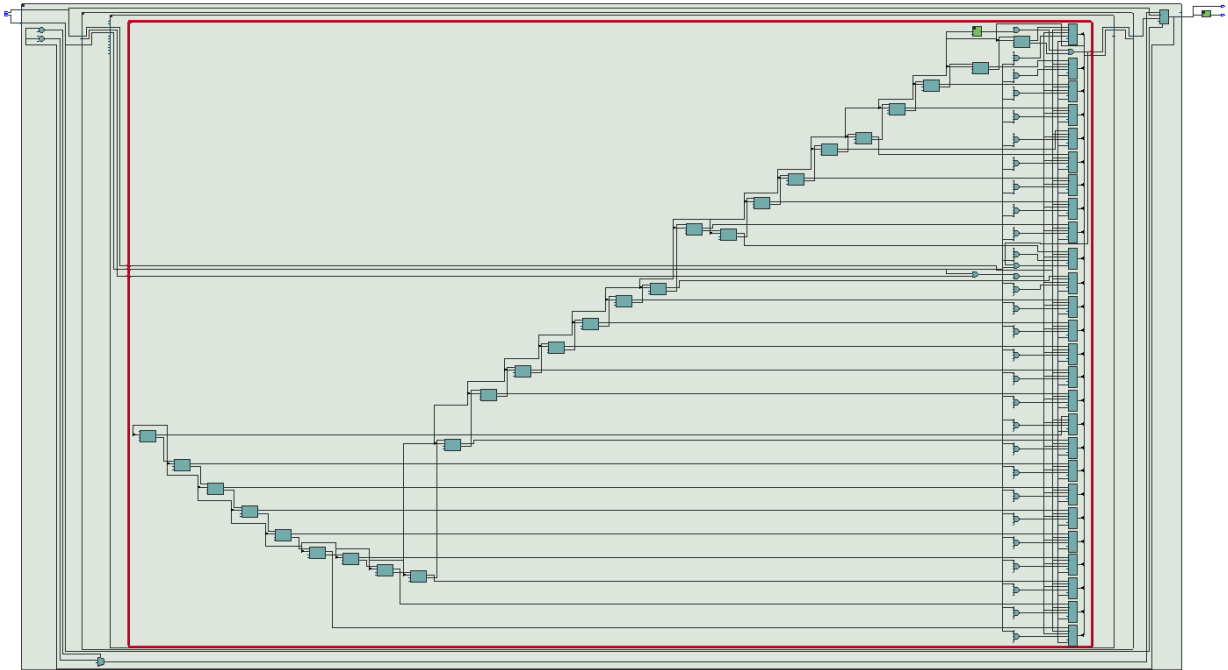


Figure 11 - This is all that it takes to count 50 million times a second?

```

module second_counter(
    input  wire clk,
    input  wire clr,
    output reg [3:0] Q,
    output wire [6:0] HEX0
);

    wire one_sec;

    LPM_Counter_50MHz Counter_50MHz (
        .clock    (clk),
        .cnt_en   (1'b1),
        .sclr     (clr),
        .cout     (one_sec),
        .q()
    );

    always @(posedge clk or posedge clr) begin

```

```

        if (clr) begin
            Q <= 4'd0;
        end else if (one_sec) begin
            if (Q == 4'd9) begin
                Q <= 4'd0;
            end else begin
                Q <= Q + 1'b1;
            end
        end
    end
end

endmodule

```

```

module LPM_Counter_50MHz (
    clock,
    cnt_en,
    sclr,
    cout,
    q);

    input    clock;
    input    cnt_en;
    input    sclr;
    output   cout;
    output   [25:0] q;

    wire sub_wire0;
    wire [25:0] sub_wire1;
    wire cout = sub_wire0;
    wire [25:0] q = sub_wire1[25:0];

    lpm_counter LPM_COUNTER_component (
        .clock (clock),
        .cnt_en (cnt_en),
        .sclr (sclr),

```

```

        .cout (sub_wire0),
        .q (sub_wire1),
        .aclr (1'b0),
        .aload (1'b0),
        .aset (1'b0),
        .cin (1'b1),
        .clk_en (1'b1),
        .data ({26{1'b0}}),
        .eq (),
        .sload (1'b0),
        .sset (1'b0),
        .updown (1'b1));
defparam
    LPM_COUNTER_component.lpm_direction = "UP",
    LPM_COUNTER_component.lpm_modulus = 500000000,
    LPM_COUNTER_component.lpm_port_updown = "PORT_UNUSE
D",
    LPM_COUNTER_component.lpm_type = "LPM_COUNTER",
    LPM_COUNTER_component.lpm_width = 26;

endmodule

```

Part 5

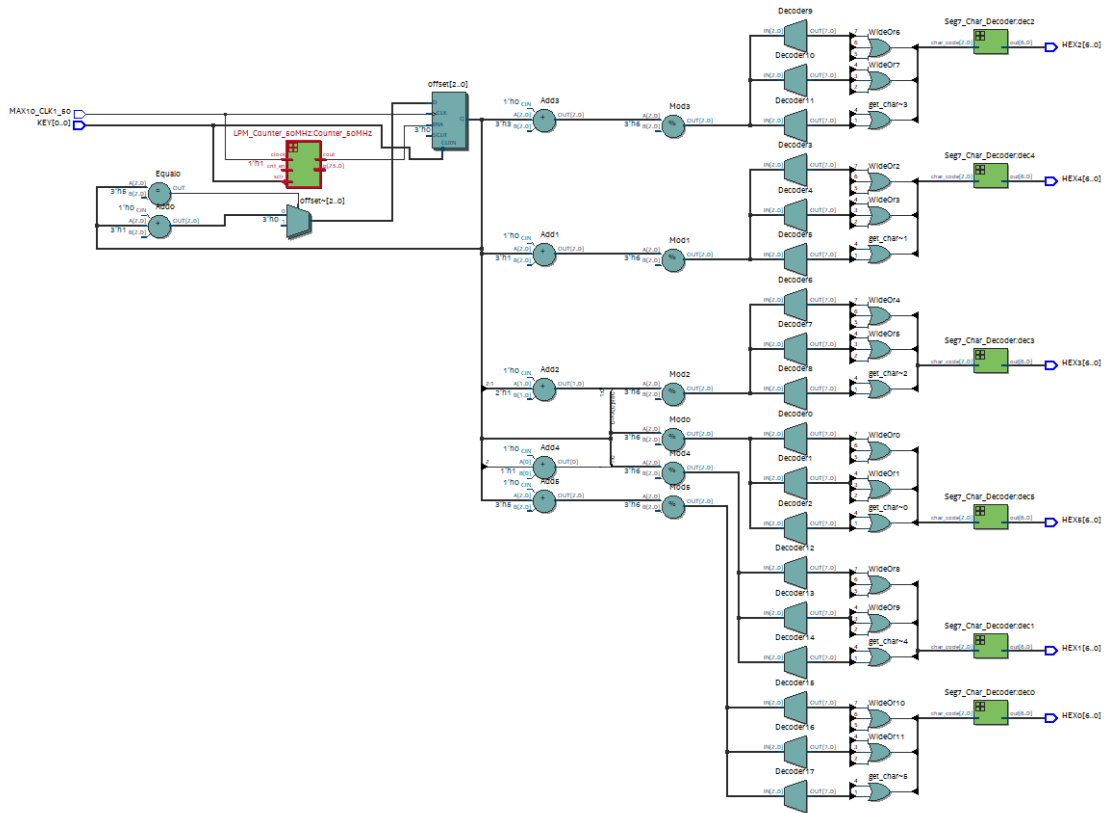


Figure 12 - RTL view of HELLO Ticker Tape circuit.

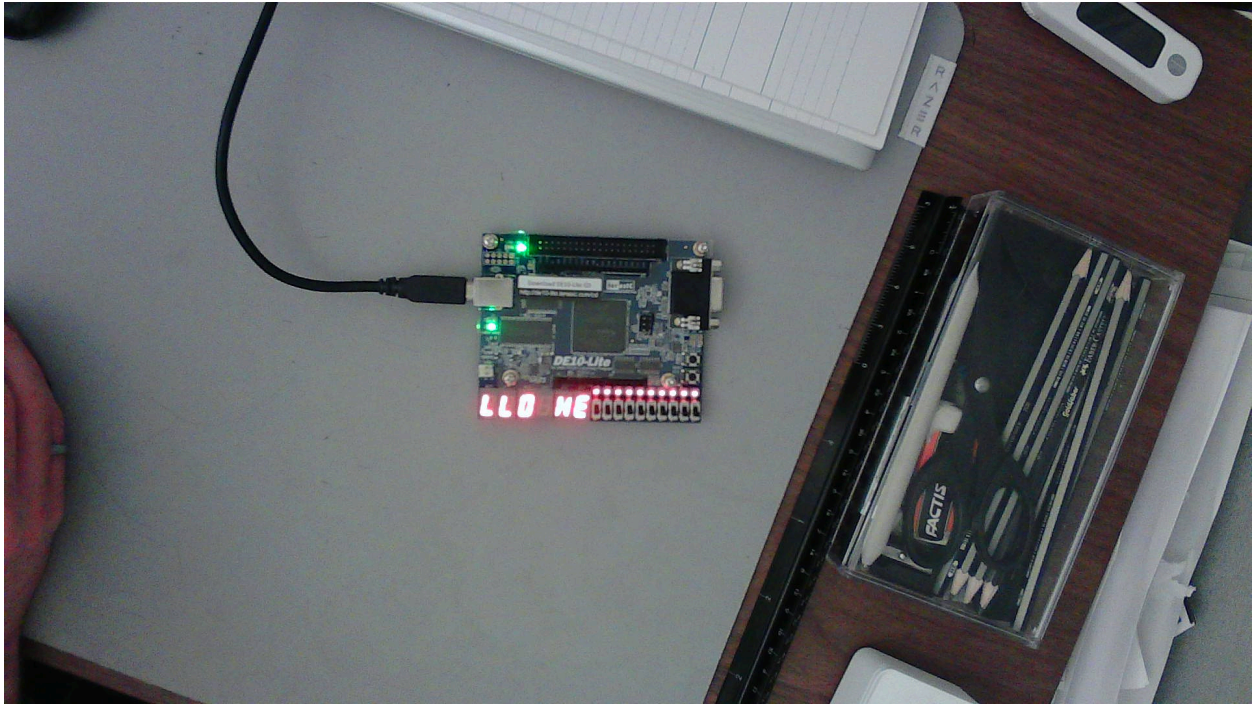


Figure 13 - Hello Ticker tape using quartus

```
module Lab_4 (  
    input wire MAX10_CLK1_50,  
    input wire [0:0] KEY,  
    output wire [6:0] HEX0,  
    output wire [6:0] HEX1,  
    output wire [6:0] HEX2,  
    output wire [6:0] HEX3,  
    output wire [6:0] HEX4,  
    output wire [6:0] HEX5  
);  
  
    wire clr = ~KEY[0];  
    wire one_sec;  
    reg [2:0] offset;  
  
    LPM_Counter_50MHz Counter_50MHz (  
        .clock (MAX10_CLK1_50),  
        .cnt_en (1'b1),
```

```

        .sclr (clr),
        .cout (one_sec)
    );

    always @(posedge MAX10_CLK1_50 or posedge clr) begin
        if (clr) begin
            offset <= 3'd0;
        end else if (one_sec) begin
            if (offset == 3'd5) begin
                offset <= 3'd0;
            end else begin
                offset <= offset + 1'b1;
            end
        end
    end
end

function [2:0] get_char;
    input [2:0] pos;
    begin
        case (pos % 6)
            3'd0: get_char = 3'd0;
            3'd1: get_char = 3'd1;
            3'd2: get_char = 3'd2;
            3'd3: get_char = 3'd2;
            3'd4: get_char = 3'd3;
            3'd5: get_char = 3'd4;
            default: get_char = 3'd4;
        endcase
    end
endfunction

Seg7_Char_Decoder dec5(
    .char_code(get_char(offset + 3'd0)),
    .out(HEX5)
);
Seg7_Char_Decoder dec4(

```

```

        .char_code(get_char(offset + 3'd1)),
        .out(HEX4)
    );
    Seg7_Char_Decoder dec3(
        .char_code(get_char(offset + 3'd2)),
        .out(HEX3)
    );
    Seg7_Char_Decoder dec2(
        .char_code(get_char(offset + 3'd3)),
        .out(HEX2)
    );
    Seg7_Char_Decoder dec1(
        .char_code(get_char(offset + 3'd4)),
        .out(HEX1)
    );
    Seg7_Char_Decoder dec0(
        .char_code(get_char(offset + 3'd5)),
        .out(HEX0)
    );

endmodule

```

```

module Seg7_Char_Decoder (
    input  wire [2:0] char_code,
    output reg  [6:0] out
);

    always @(*) begin
        case (char_code)
            3'd0: out = 7'b000_1001;
            3'd1: out = 7'b000_0110;
            3'd2: out = 7'b100_0111;
            3'd3: out = 7'b100_0000;
            3'd4: out = 7'b111_1111;
            default: out = 7'b111_1111;
        endcase
    end
endmodule

```

```
        endcase  
    end  
  
endmodule
```